

The Agentic Company OS: Substrate Inversion for Sustained Enterprise Agent Deployment

Oliver Aleksander Larsen¹[0009-0003-6944-2560]*
 Mahyar T. Moghaddam¹[0000-0001-5028-7546]

SDU Software Engineering, University of Southern Denmark, Odense, Denmark
 olar@mmi.sdu.dk*, mtmo@mmi.sdu.dk

Abstract. Enterprise AI agents often succeed in a demonstration and then stall once they must operate day after day. An industry report estimates that most pilots never reach production and that deployed systems rarely retain feedback or improve over time, while agent benchmarks show single-run successes masking unreliable repetition. We argue that these failure modes share a common architectural root: agents reason over data structured for human operators and traditional applications, not for the language models that power them. This position paper proposes that companies deploying agents in sustained operation should rebuild their cognitive substrate, the shared environment agents read as working context, around representations matched to that reasoning surface, isolating schema translation to the action boundary. Markdown is the instantiation available today, not a proven agent-native primitive. Two mechanisms ground the argument: context-bandwidth asymmetry, the gap between one-pass reading of connected prose and field-by-field typed access that strips relations; and cross-loop coupling, the claim that action, skill, and policy loops compound only if they share one substrate. A four-layer framework (Data, Knowledge, Intelligence, Governance) operationalizes the position, with a Sync Agent enforcing the action boundary and a per-skill trust gradient, making governance and auditability structural properties of the substrate. The position revives the shared-substrate tradition of classical multi-agent systems under LLM-era economics. We analyze the main objections and risks, including indirect prompt injection on the compile path, and outline a research agenda for evaluating substrates directly.

Keywords: LLM agents · Agentic AI · Agent governance · Enterprise architecture · Business process management · Prompt injection · Multi-agent coordination

1 Introduction

Enterprise AI adoption is scaling rapidly [24], and agentic systems are entering production deployment [39]. Yet a widely circulated industry report estimates that only a small fraction of enterprise GenAI pilots reach production [1], and

the single successful run of a demonstration masks inconsistency that repeated operation exposes [46]. We identify five recurring failure modes, often treated in isolation: governance gaps, where an agent acts outside policy; transparency gaps, where reviewers cannot reconstruct agent actions; coordination gaps, where agents drift out of step; safety gaps, where untrusted content contaminates trusted reasoning; and plateaued improvement, where deployments stop improving despite model upgrades. An index of deployed agents documents how sparsely governance, transparency, and safety practices are disclosed [39]. The industry report also finds that most deployed GenAI systems fail to retain feedback or improve over time [1], and model version changes can even degrade behavior [2]. The motivating hypothesis is that this stall is architectural: it hardens into a plateau unless the agent’s working context is rebuilt.

We propose that these are not five orthogonal problems but five symptoms with a common architectural root: agents reason over a substrate, the representation an agent reads as working context, built for human operators and conventional applications, not for the language models doing the reasoning. Rebuilding the substrate around the agent’s reasoning surface would address four of the five directly (governance, transparency, coordination, and plateaued improvement); the safety gap it reshapes rather than removes (Sect. 6.1).

We argue that companies deploying AI agents in sustained operation should rebuild their cognitive substrate (the shared environment the agent reads as working context) around representations matched to that reasoning surface, with schema translation isolated to the action boundary. The position is a structural commitment about where agent reasoning happens, not a framework, runtime, or model-class recommendation. Markdown is the instantiation available today, not a proven agent-native primitive (matched to how an LLM reads connected prose rather than typed application schemas). Any representation better matched to an LLM’s reasoning surface would serve equally well. We do not claim this architecture has been shown to outperform typed-tool, retrieval, or hybrid stacks; Sect. 7 specifies the comparison that would. We call this commitment substrate inversion. It renews the shared-substrate tradition of classical multi-agent systems under LLM-era economics, a lineage that Sect. 2 develops.

Three trends make the argument timely. First, one million tokens of LLM context [3] is on the order of two thousand pages of Markdown, enough to hold the compiled knowledge base of a small or mid-sized company. Second, closed-loop measurement is becoming cheap: a thousand outcome records fit in a single context window, and an LLM can draft a playbook revision from them in one pass. Third, deployments are moving past chatbot scope into autonomous operation, so substrate choices can no longer be deferred.

The paper makes three contributions, with relevance to business process management, contextual decision-making, and applicative risk identification. First, it states the substrate-inversion position, grounded in two mechanisms: context-bandwidth asymmetry (one-pass connected prose versus field-by-field typed access that strips relations) and cross-loop coupling (action, skill, and policy

loops compound only if they share one substrate). Second, it presents a four-layer framework (Data, Knowledge, Intelligence, Governance) operationalizing the position, with a Sync Agent at the action boundary, a per-skill trust gradient, and three closed improvement loops sharing one substrate. Third, it critically examines the strongest objections (Sect. 5) and risks, including indirect prompt injection (Sect. 6.1), deriving an agenda for evaluating substrates rather than agents, with falsifiable hypotheses and an experiment design to test them (Sect. 7). The applicable domain is enterprise operations where interpretation-heavy work dominates. The framework distills an ongoing design effort for an agent-native company operating system, presented as an arguable architectural thesis open to empirical test.

Section 2 situates the position in classical multi-agent systems and in three families of current enterprise agent stacks. Section 3 states the two mechanisms and the hypotheses they yield. Section 4 presents the four-layer framework and walks through a constructed one-page customer file that both a human and an agent read as shared context (Sect. 4.5). Section 5 takes three alternative approaches at their strongest; Sect. 6 names imported risks and scope conditions; Sect. 7 gives an adoption path and a substrate-controlled benchmark.

2 Background and Related Work

Software agents that perceive, reason, and act have been studied since the 1980s, from blackboard systems such as Hearsay-II [8] through the mid-1990s surveys that named the field [45]. What changed with large language models is not the idea of an agent but the form of its working context: a natural-language window rather than a hand-engineered memory. Enterprise agent stacks built on that surface fall into three families. Retrieval-augmented agents over relational systems [10,21] retrieve fragments from CRMs, document stores, and knowledge bases and prompt the model with them. The working context still comes from the relational store. Tool-using agents on typed schemas compose function calls into multi-step workflows through ReAct-style loops [47]. Here the working context still comes from the schema-locked system behind the typed contracts (fields and relations fixed by the application’s type system). Vertical AI applications ship domain-specific copilots (support, sales, legal, finance), each with a store optimized for its vertical. All three families deliver value on bounded tasks, and benchmark evidence is consistent with a ceiling tied to the working context, not only to the model: GAIA performance drops with task depth [26], and AgentBench [23] reports persistent multi-step gaps, both confounded with planning and tool limits. A recent file-native evaluation finds serialization format alone does not significantly affect single-task accuracy [25]; what no benchmark yet isolates is substrate structure (connected prose versus fragmented access) or improvement that compounds over repeated operation, which motivates the evaluation proposed in Sect. 7. The first two families inherit the working context from the human-first system the agent was attached to, and vertical applications choose theirs only per vertical. Translation compensates for the resulting mismatch but does not

Table 1. Where an agent reasons, where schema translation is paid, whether improvement loops share a substrate, and where audit and governance live. The three deployed families inherit a human-first store and keep policy and provenance outside it; substrate inversion rebuilds the working context and places translation, audit, and policy on one versioned corpus (Sect. 3).

	RAG agents	Typed-tool agents	Vertical apps	Substrate inversion
Reasoning surface	retrieved fragments	typed results	vertical schema	compiled corpus
Schema translation	retrieval pipeline	every call	app adapters	action boundary
Loops on one substrate	none	action only	per vertical	all three
Audit-trail locus	source logs	API logs	vendor logs	version control
Governance locus	external engine	external engine	vendor controls	substrate itself

remove it (Table 1). We reject that inheritance: the working context should be rebuilt for the agent, not borrowed from the human-first system and patched with retrieval or tools.

Three theoretical anchors support the position. Sculley et al. [37] describe hidden technical debt in machine learning systems as glue code, pipeline jungles, and feedback loops that age badly. Translation layers between agents and operational ground truth are the agent-era form of that debt. Conway’s observation that organizations produce systems mirroring their own communication structure [4] has a direct agent-era reading: agent organizations inherit the schema fragmentation of the org chart. Hutchins’s analysis of distributed cognition [17] argues that multi-actor cognition runs through shared external representations. The position extends this claim from coordination to closed-loop improvement, because corrections travel along the representations that already coordinate human and agent collaborators.

The architecture family itself predates the LLM. Blackboard systems coordinated specialist problem solvers through a shared working memory [8,28]. Classical MAS explored shared media and institutions (Linda, environment-mediated coordination, electronic institutions, adjustable autonomy, and mediators [11,43,9,35,44]) over two decades. These designs were structurally right and economically blocked: every shared representation had to be hand-engineered into frames, tuples, or ontologies before any agent could read it. The other classical blocker was the blackboard control problem, deciding which knowledge source should act next [28]. The LLM removes the first blocker and relocates the second: plain prose is now machine-readable at the scale of a company, while scheduling falls to an orchestrator agent whose control policy remains open (Sect. 8). That reversal makes the blackboard pattern economically plausible as enterprise infrastructure.

Several recent artifacts inform the framework. Karpathy’s living-wiki proposal [18] gives the substrate idea its current instantiation: raw data feeds an LLM compiler that emits and maintains a wiki of Markdown files, which the model navigates index-first, reading pages as needed instead of retrieving fragments at query time. LLM-based multi-agent systems, surveyed by Guo et al. [13], provide scaffolding for early prototypes. Agent memory systems [30] solve the persistence problem inside the runtime; substrate inversion solves it in the shared substrate. Practitioners in business process management report demand for governance guardrails on agents [41].

The position engages the agent-safety and prompt-injection literature in two directions. In the first, it contributes safety infrastructure: the audit trail is the version-control log, the trust gradient is recorded in versioned skill metadata, and policy lives in the same Markdown the agents read. In the second, it imports a genuine threat surface: compiling untrusted external content into trusted agent context exposes the architecture to prompt-injection attacks [12,33], with indirect injection as the most serious case. Section 6.1 examines this risk.

Recent work treats one piece of the classical assembly at a time: a governance regime [38], a learning method [42], a safety control [15], and blackboard coordination itself, revived as a runtime pattern for LLM multi-agent systems [14,34]. To our knowledge, no prior or concurrent work assembles the whole: an enterprise prose substrate, governance and audit in version control, and improvement loops coupled through one corpus.

3 Substrate Inversion: The Argument and Its Mechanisms

The position commits to two structural properties: a single agent-native reasoning surface, and schema translation isolated to the action boundary. The two mechanisms below establish why these properties, not the implementation details around them, are the load-bearing choices. The multi-agent systems community has long treated the environment as a first-class abstraction [43]. Substrate inversion applies that argument to LLM agents: the substrate is the environment, and choosing it for the agent, rather than inheriting it from the legacy application, is the central design decision.

Mechanism 1: context-bandwidth asymmetry. Modern LLM context windows accept roughly 10^6 tokens of unstructured prose [3], consumed in a single pass with relational context carried implicitly. Typed APIs and relational schemas expose the same information field by field, with relations declared in the schema rather than carried in the content (Fig. 1). Each schema crossing therefore strips relational context that the agent must rebuild in working memory. Consider a multi-step sales decision that depends on five contexts: the prospect’s most recent reply, the deal’s pipeline stage, the customer’s payment history, the prospecting playbook in force, and the company’s pricing rules. On a Markdown substrate these arrive as a connected narrative read in one pass. Routed through a CRM, a ledger, and an email API, they arrive as typed records, and the agent reassembles the narrative anew at each step.

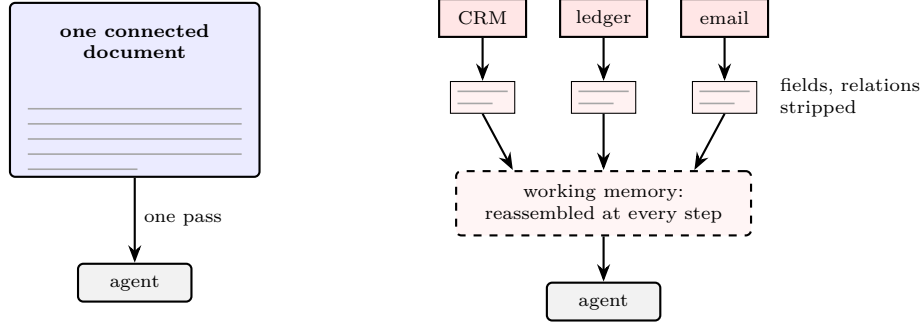


Fig. 1. Context-bandwidth asymmetry in plain terms: on the left, the agent reads one connected document in a single pass, with relations carried in the prose; on the right, typed systems return fields that strip those relations, so the agent rebuilds the narrative in working memory at every step. Hypothesis H1 predicts that this reassembly cost bounds compositional reach.

The loss is depth-dependent: the deeper the compositional task, the more the per-step reassembly compounds. Even information present in context is recovered unreliably depending on its position, a limit one-pass reading shares [22]. We state the resulting prediction as hypothesis H1: per-step reassembly cost bounds an agent’s compositional reach, a bound that tightens with task depth. The benchmark in Sect. 7 tests it.

Mechanism 2: cross-loop coupling. We model sustained improvement as three feedback loops at three time-scales: an action loop on seconds (act, observe, adjust), a skill loop on days (accumulated outcomes revise the playbook), and a policy loop on weeks or months (humans or governance agents detect systemic patterns and change the rules). The loops are coupled: a corrected outcome compounds only if it propagates from action to skill to policy, and a policy update takes effect only when the next action reads it. The loop structure echoes autonomic computing’s MAPE-K and the three-layer architecture of self-managed systems [19,20]; substrate inversion externalizes the knowledge component as a prose corpus that humans, agents, and governance share. Enterprise stacks often split them across three substrates: the action loop in agent memory, the skill loop in fine-tuning artifacts or vendor prompt registries, and the policy loop in meeting notes. A study of teams building product copilots documents fragmentation of this kind [32], echoing the technical debt long described for classical ML [37]. A correction noticed at action time then fails to migrate to skill time, a policy decision fails to alter the next call, the loops desynchronize, and corrections stop compounding.

Linda-style coordination models decouple writers and readers in time and identity [11]; cross-loop coupling exploits the same decoupling, since a policy written at week scale is read at action scale. A single Markdown substrate is the kind of shared external representation that distributed cognition theory [17] describes: the playbook the agent reads next is the policy committed in the last

governance review. This yields hypothesis H2, the paper’s central falsifiable claim: under split substrates, improvement decays toward the noise floor rather than compounding as it does on a unified substrate, and model upgrades alone do not rescue it. The same benchmark in Sect. 7 tests it.

Together the two mechanisms point to the full substrate-inversion commitment: one representation matched to the LLM consumer, shared across the three improvement loops, with translation paid once at the periphery.

4 The Agentic Company OS Framework

The four layers exist because each has a job the others cannot absorb. The Data Layer stays the company’s authoritative systems of record: transactional integrity and statutory record-keeping are orthogonal to reasoning, and demanding both from one store is what produces schema fragmentation. The Knowledge Layer is the reasoning surface: compiled, connected prose that humans and agents read as working context. The Intelligence Layer coordinates by writing those shared files, so there is no out-of-band message bus. The Governance Layer lives in the same file, so policy cannot drift from operational state. The Sync Agent exists for one reason: it is the only translation at the action boundary, compiling into the Knowledge Layer and executing typed writes back out. A trust gradient (Sect. 4.3, Fig. 3) makes governance structural, and three closed loops (Sect. 4.4) compound through the shared substrate (Sect. 4.1, Fig. 2).

Open operational limits, including freshness of compiled files, read-level authorization, write concurrency, cost and latency of rereading, and recovery from substrate corruption, are design caveats rather than solved claims; Sect. 7 lists them as questions a deployment must answer.

4.1 The Four Layers

Data Layer. The Data Layer is the infrastructure the company already runs: the payment processor, the CRM, the accounting system, the calendar, the email server. These systems are schema-locked. Agents visit them only through the Sync Agent, when an action needs the guarantees of statutory record-keeping or an investigation needs the raw records. The framework leaves these systems authoritative and unmodified, because their job (transactional integrity, regulatory compliance) is orthogonal to the agent’s job (reasoning), and demanding both from one substrate is what produces schema fragmentation. The integration glue (webhooks, credentials, write-backs) relocates into the Sync Agent rather than disappearing.

Knowledge Layer. The Knowledge Layer is a single Git repository of Markdown files, organized as a wiki with bidirectional links and an LLM-compiled knowledge graph [7], in the spirit of recent agent-native proposals [18]. Decisions, summaries, playbooks, and contextual narrative live here; raw transactional rows do not. Humans and agents read the same files and leave their traces in the same Git history. In agents-and-artifacts terms [29], customer files, playbooks,

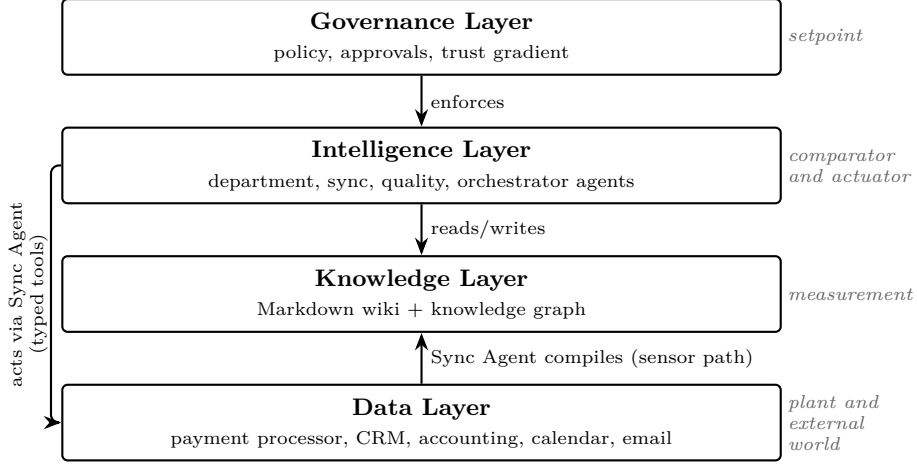


Fig. 2. The four-layer architecture as a closed control loop. Governance holds the setpoints, Intelligence compares and acts, Knowledge is the measurement substrate, and the Sync Agent is the only translation at the action boundary (sensor and actuator). Action, skill, and policy loops compound only because they read and write the same Knowledge Layer (cross-loop coupling).

and policy documents are coordination artifacts: first-class entities that mediate agent work and persist beyond any single interaction. This is the reasoning surface hypothesized by Mechanism 1, not a measured gain over retrieval, and it stays small: at one page per customer and per playbook, a 50-person company with a few hundred accounts compiles to a few hundred thousand tokens, inside one context window. Beyond that envelope, at thousands of accounts, one-pass reading gives way to index-first navigation over the same substrate, and the bandwidth argument applies per page set: each page is a connected narrative, not a stripped fragment.

Intelligence Layer. The Intelligence Layer is a small set of specialized agents (department, sync, quality, orchestrator) that share the Knowledge Layer as working memory. Agents exchange no out-of-band messages; they coordinate by writing to shared files, and the substrate serves as the message bus. This revives the blackboard and tuple-space patterns [11,28], with one decisive change: the shared workspace is prose that the LLM and the human read natively, not an engineered representation that only purpose-built knowledge sources could parse. Write concurrency follows a minimal regime: the Sync Agent alone writes compiled frontmatter (the YAML header it compiles), the skill loop revises playbook bodies while trust fields accept no LLM-agent writes (a commit-gate rule; Sect. 4.3), agents append to activity logs rather than editing prose in place, and the orchestrator serializes residual same-file edits.

Governance Layer. The Governance Layer is the policy plane: rules, approval thresholds, and the per-skill trust gradient (Sect. 4.3), all written in the

same Markdown the agents read. Every commit is signed and revertable, and every policy change is itself an artifact in the audit trail. No separate policy engine can drift from operational state, because the policy engine is the operational state, echoing law-governed interaction [27] and the normative rules of electronic institutions [9]. Legibility is in-band by construction; enforcement sits at the action boundary (Sect. 6.1).

4.2 The Sync Agent and the Action Boundary

The Sync Agent is the only agent permitted to call external systems, playing the classical mediator role [44] at the action boundary. Two ingestion modes feed the Knowledge Layer. *Continuous ingestion* combines webhooks for structured events with a single forwarding email address that catches the unstructured long tail (meeting notes, ad-hoc updates). *Batch ingestion* is a one-time migration at onboarding: a script pulls historical data through existing read-only API keys and writes summary files into the right folders. Continuous ingestion keeps the Knowledge Layer fresh without polling, and quality agents reconcile compiled files against the systems of record via the Sync Agent, flagging missed events and hallucinated summaries for recompilation: compiled state is a materialized view, never authoritative.

The Sync Agent writes summaries in the Knowledge Layer’s representation, not raw rows in the source schema. A customer with dozens of invoice lines and activity entries in the CRM appears in the substrate as a single one-page file. The full rows remain in the systems of record, where law and audit need them. An agent that requires a deep investigation calls typed tools via the Sync Agent and writes a fresh summary back afterwards. The Model Context Protocol, surveyed in [16], standardizes typed tool contracts at this boundary, at the periphery, not in the reasoning loop. Recompiling a few-hundred-thousand-token Knowledge Layer daily costs single-digit dollars on a mid-tier long-context model; continuous reasoning likely dominates the budget, since rereading the full corpus on every action multiplies that cost and adds prefill latency to the seconds-scale action loop. Prefix caching and selective reads are the levers, and selective reading trades against the one-pass property of Mechanism 1.

4.3 Trust Gradient and In-Band Governance

The central governance instrument is the *trust gradient*: a per-skill trust level encoded in the skill’s Markdown frontmatter. Four levels define what the agent may do (Fig. 3). At **shadow** the agent drafts an action and never executes; at **assist** a human approves each draft before execution; at **autonomous-sampled** the agent executes and a human spot-checks a sampled fraction; at **full** the agent executes freely under retrospective audit.

Trust attaches to skills, not to agents. A sales agent may run `qualify_lead` at full autonomy while holding `negotiate_contract` at shadow on the same call, the pairing shown in Fig. 3. Sensitive actions (legal commitments, refunds above a threshold, hiring decisions) stay gated regardless of trust elsewhere. Trust fields

accept no LLM-agent writes. A deterministic controller may downgrade a skill after a severe regression, logged as a Git commit; only a human principal may raise a level. How outcomes should otherwise move a skill’s level is open. The gradient is a substrate-encoded form of adjustable autonomy [35] and compresses classical levels-of-automation scales [31]. The new element is that the autonomy level is a versioned artifact in the repository the agent reasons over, so every change of autonomy is itself an auditable commit.

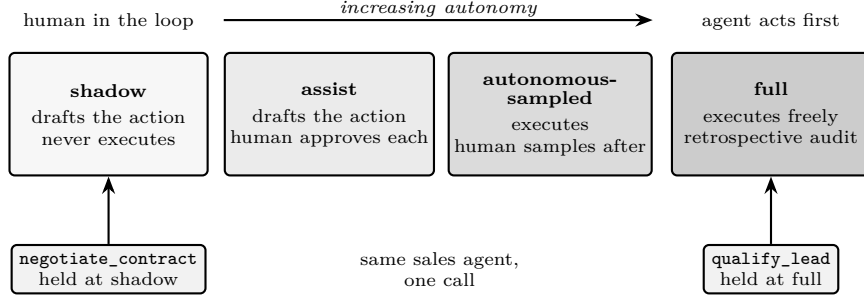


Fig. 3. The per-skill trust gradient. Four rungs define what a skill may do, from drafting with no execution (shadow) to unsupervised execution under retrospective audit (full). Trust attaches to skills, not to agents: one sales agent can hold **qualify_lead** at full and **negotiate_contract** at shadow on the same call. A human principal raises a level; a deterministic controller may lower it after a severe regression, logged as a signed commit.

4.4 Three Closed Loops, One Substrate

Table 2. Three closed loops at three time-scales, sharing one Markdown substrate.

Loop	Scale	What closes it	Substrate artifact
Action	seconds	Outcome compared to intent	Outcome appended to log file
Skill	days	Trust gradient, runtime learning	Playbook revised, trust fields excepted
Policy	weeks	Human or governance review	Policy file updated in review

Many company processes are open loops by accident: a sales reply is sent and the outcome never examined; a refund is issued and never audited. LLMs change the economics of closing such loops.

The four layers map onto control roles (Fig. 2): the Governance Layer holds the setpoints, the Intelligence Layer is comparator and actuator, the Knowledge

Layer is the measurement substrate, and Sync Agent ingestion is the sensor pathway. The three loops in Table 2 close at different time-scales but read from and write to the same files, so one corrected outcome serves all three: it is the next action’s input, the next skill revision’s evidence, and the next policy review’s pattern. This is stigmergic coordination [40,43]: a correction propagates because later work reads the marks earlier work left in the substrate. This compounding is cross-loop coupling (Mechanism 2) at work. Coupled loops that close cheaply and quickly can also oscillate: a skill loop may chase action-loop noise, and a policy loop may institutionalize a transient. The framework’s dampers are the trust gradient’s graduated rollout, sampled human review, and the slower policy cadence.

4.5 Worked Example: A Customer File

The following customer file is a constructed illustration, not deployment data.

```

---
status: active
last_payment: 2026-05-01
lifetime_value_eur: 38400
plan: annual
owner: Sara Chen
---

# Acme Corp

Annual plan since March 2026; renewal on track.

## Activity log
- 2026-05-01: payment EUR 4,872 received.
- 2026-03-15: upgraded to [[Plans/Annual]].

```

Both the sales agent and a human account owner read the same file. The wiki links make it a node in the knowledge graph, and the Sync Agent updates the frontmatter whenever external state changes.

5 Alternative Approaches

For each of three alternative positions on agent infrastructure, we present the strongest version of the case, respond from the mechanisms of Sect. 3, and state where we concede ground.

5.1 Better Retrieval Is Sufficient

The case. Retrieval-augmented generation [21] over enterprise data stores has matured rapidly: chunking, query rewriting, query expansion, learned re-rankers,

and iterative retrieval close many early gaps [10]. On this view retrieval, not substrate, is the bottleneck: agents reason over existing company data while the systems of record stay untouched, and better retrieval is the cheaper, more incremental path to the same end state.

Response. Substrate inversion is not an argument against retrieval. Retrieval returns fragments that the agent must reassemble in working memory at every step. On H1, raising retrieval quality lifts the ceiling without removing it: better-chosen fragments are still fragments. The framework retains retrieval as a Sync Agent tool for raw records, not as the substrate. We concede that at low compositional depth the two approaches are functionally equivalent, and the choice between them is economic rather than architectural.

5.2 Tool-Using Agents on Typed Schemas

The case. With function-calling interfaces [36], ReAct-style reasoning loops [47], and well-typed tool contracts, agents compose typed actions into multi-step workflows without rebuilding the substrate, and the typed contract between agent and tool gives auditors a precise review surface. The multi-agent-systems community has invested decades in typed agent communication languages and shared ontologies [45]. On the strongest reading, typed schemas are the solution: better tool design and training will erase the asymmetry at far lower cost than unifying everything in Markdown.

Response. Our position keeps typed schemas as the right tool at the action boundary. They do not, however, erase the asymmetry: the lossy case of Mechanism 1 is a decision routed field by field through a CRM, a ledger, and an email API, and better typing removes per-call ambiguity, not per-step reassembly. On H1, well-typed records are still fragments. The deeper disagreement concerns cross-loop coupling (Mechanism 2). Tool-using agents perform well within the action loop but typically distribute the skill and policy loops across the separate substrates that Sect. 3 described, so corrections in one loop fail to propagate to the others. The strongest variant of the objection is a hybrid that versions playbooks, policies, and outcome logs in one repository while reasoning-time reads stay tool-mediated. This captures cross-loop coupling, and the residual disagreement is then exactly Mechanism 1, the factor the benchmark in Sect. 7 isolates. A second variant closes the skill loop in weights by outcome-driven fine-tuning; such loops are illegible to the policy loop, reset across model versions, and instantiate the split-substrate arm of the same benchmark. We concede that at low compositional depth tool-using agents are competitive; without the hybrid, the position is predicted to become decisive at the skill-loop and policy-loop time-scales.

5.3 Vertical AI Applications

The case. On this view, domain-specific applications (verticalized customer support, legal copilots, finance operations) capture value through deep specialization:

each vertical builds its own optimized substrate, evaluation suite, and compliance story, and enterprise buyers prefer a finished workflow to a general substrate.

Response. A company running five vertical applications operates five disconnected substrates, each with its own data model, audit trail, and governance plane: the very fragmentation that cross-loop coupling predicts will impede company-level improvement. Vertical applications nonetheless fit within our framework, since they can plug into the Intelligence Layer as specialized agents that read and write the shared substrate. Single-vertical companies are well served by such applications; we concede the point. The position targets cross-functional coordination.

6 Risks, Governance, and Scope

6.1 Risk Identification, Evaluation, and Accountability

Three concerns recur: identifying risk, evaluating it, and accounting for autonomous action. Substrate inversion makes governance and audit structural; safety is not. Safety requires active defense, and one risk is imported rather than removed.

Risk identification: the substrate changes the threat model. The compile pass turns untrusted external content into trusted agent context, a carrier of prompt injection [33], indirect injection in particular [12]; the architecture sharpens this threat beyond the per-call case. Persistent substrate poisoning: a payload compiled once is re-read by every agent on every call, so injection is durable. Cross-agent propagation: the substrate is the message bus, so one poisoned artifact reaches every agent that reads it. Trust-gradient manipulation: trust fields accept no LLM-agent writes, so the remaining attack is a poisoned compile or a bypassed commit gate, not routine agent edits. Loop laundering: a poisoned outcome can steer the skill loop’s playbook revision. Exfiltration: an injected directive can route data out through authorized write channels. And the forwarding email address is an internet-facing, unauthenticated write path into trusted context. The Sync Agent concentrates the surface: the component compiling the rawest input also holds every external credential. Privilege separation is the response: an uncredentialed compile worker writes quarantined drafts, while a credentialed deterministic action worker executes and checks trust. The compile pass needs quarantine, parameter validation, and directive detection, drawing on spotlighting [15]; this compile-path screening never finishes. Retrieval stores can also screen at ingestion; here the screening point is single, versioned, and auditable, and a miss persists substrate-wide. The single shared reasoning surface co-locates control artifacts and compiled untrusted content: policy files and untrusted summaries sit in the same repository. CaMeL [6] keeps those channels apart; the substrate as specified does not, so it needs trust-zone partitioning (provenance or taint labels on compiled artifacts), an open tension.

Risk evaluation: the trust gradient as runtime control. Per-skill trust levels make risk measurable at runtime; gating and downgrade operate as in

Sect. 4.3 and Fig. 3. In-band policy provides legibility and auditability. It cannot, by itself, enforce against a non-compliant or injected model. Enforcement therefore sits outside the LLM, at the action boundary: the Sync Agent’s monopoly on external calls lets deterministic code in the call path check a skill’s trust level when the action executes. Two conditions harden the check: only designated principals may write Governance Layer files, with one exception (a deterministic controller may lower a skill’s trust level), and the check reads from a pinned, verified revision, not the working tree. The check gates the skill, not its payload; egress control remains open.

Ethical and legal accountability: audit by construction. Rules, approval thresholds, and trust levels are first-class substrate artifacts; no separate rule engine drifts out of date. Git history serves as the audit trail: every change, human or agent, is diffable, timestamped, and revertible, and an auditor reads the same files the agent reads. Attribution and immutability hold only under signed commits and protected branches, controls the deployment must enforce. Two open problems remain. Read-side access control: the gradient gates actions, nothing yet gates reads in one shared repository where a CRM would enforce row-level permissions; per-directory ACLs or trust-zone sharding are candidate directions. Erasure versus immutability: the worked example stores customer data, and deletion under data-protection law conflicts with the signed history the audit rests on, a conflict crypto-shredding of per-record keys would blunt.

6.2 Scope: Where the Position Fails

The position is best suited for interpretation-heavy operations, where the bottleneck is the agent’s grasp of context and the substrate’s job is to keep that context legible. It loses for high-frequency transactional systems (algorithmic trading, ad bidding, payments routing), where typed-state guarantees and tight latency budgets dominate. Nor does the position eliminate structured systems of record: the payment processor still holds the money, and the ledger remains authoritative.

7 Research and Adoption Agenda

Adoption path. Start with one department, not a company-wide cutover. Pick a workflow that is interpretation-heavy and already has a named owner (sales qualification, support triage, and contract review are typical), migrate that workflow into Markdown files the department agent reads on every call, and wire the Sync Agent only to that department’s external systems. Run the agent up the trust gradient of Sect. 4.3 (Fig. 3): shadow, then assist, then autonomous-sampled, then full, with a human principal required for every increase. The headline metric is time-to-full-autonomy, checked against regression rate and rollback rate.

Rollback is a first-class requirement. Because the Data Layer remains authoritative, reverting the department means resetting the skill’s trust level to shadow by a signed commit, redeploying the previous Knowledge Layer revision, and

leaving the CRM, ledger, and mail systems untouched. The department can fall back to its prior tools the same day. Only after the pilot holds under a rollback drill and sampled audit should the pattern replicate to a second department; cross-department workflows on the shared substrate are the prize, not the starting point.

Research agenda. The central item is a substrate-controlled benchmark specified so a reader can run it. Hold the model and the task suite fixed. Vary two factors. Representation: unstructured Markdown, a typed schema, retrieval-augmented generation over the same documents, a relational store, and a hybrid that versions playbooks in a repository while reasoning-time reads stay tool-mediated. Loop coupling: the three improvement loops of Sect. 3 share one substrate, or they are split (action in agent memory, skill in a prompt registry or fine-tune, policy in a separate store). Every condition compiles from one source corpus, so information content is held constant and only structure and coupling change.

Compositional depth is the number of inter-entity relations a task must jointly resolve (for example, a refund that depends on the latest ticket, the payment record, and the current policy), with source-fact count and total tokens held fixed. The longitudinal arm runs a skill loop: a fixed agent revises the playbook from the last k outcomes, identically across conditions, and the score is pass-rate delta per closure. The noise floor is variance under a frozen playbook. Compounding is a positive trend over tens of closures that clears that floor. Replicating the design across model tiers tests whether any plateau persists as models improve. Two further instruments are needed: structured-merge primitives for concurrent agent edits, and a compile-path injection benchmark that extends per-call suites [5] to persistence and cross-agent propagation.

H1 and H2 state what would falsify the position. H1 predicts that per-step reassembly cost bounds compositional reach, so the connected-prose condition should pull away from typed and retrieval conditions as depth grows, with source facts held fixed. H1 is false if the conditions stay within noise of each other at high depth, or if typed or retrieval conditions win. H2 predicts that under split substrates, improvement decays toward the noise floor rather than compounding as it does on a unified substrate, and that model upgrades alone do not rescue the split case. H2 is false if the split condition compounds as well as the unified one, or if a model upgrade closes the gap without unifying the substrate. Neither outcome is yet measured; both are the point of the benchmark.

Open operational questions. Even if H1 and H2 held, a deployment would still have to answer questions this paper does not resolve. Scalability: at what corpus size does index-first navigation erase the one-pass advantage of Mechanism 1, and does the per-page connectedness claim survive? Freshness: how stale may a compiled customer file be relative to the system of record, and who detects missed events? Read-level authorization: the trust gradient gates actions, not reads; what access control belongs in a shared repository where a CRM would enforce row-level permissions? Concurrency: the minimal write regime of Sect. 4.1 is a convention, not a merge semantics; what happens when two agents edit the

same playbook? Cost and latency: daily compilation is cheap on a mid-tier model, but rereading the corpus on every action is not; which caching and selective-read policy preserves Mechanism 1 without blowing the seconds-scale action loop? Recovery: if compiled files are poisoned or corrupted, what is the restore unit (file, commit, trust zone), and how fast can the Knowledge Layer be rebuilt from the systems of record? These are design caveats. Treating them as solved would overclaim.

Standards. For standards bodies, the open task is to specify substrate-level audit guarantees (audit trail by construction, in-band policy, structured trust levels) in a vendor-neutral form, so that Git-based provenance can sit alongside log-based provenance in compliance frameworks.

8 Conclusion and Future Work

Companies that deploy AI agents in sustained operation should rebuild their cognitive substrate around representations matched to the agent’s reasoning surface, with schema translation confined to the action boundary. Markdown is the instantiation available today, not a proven primitive, and superiority over typed-tool, retrieval, or hybrid stacks remains a hypothesis, not a result. The commitment concerns the reasoning surface, not the framework, runtime, or model class. Its operational form is the four-layer framework, the Sync Agent, the per-skill trust gradient, and the three closed loops. Policy legibility and a single audit surface then follow by construction. Safety and attribution additionally require enforcement at the action boundary, signed commits, and a guarded compile path (Sect. 6.1).

Five questions remain open. **(1) Threat model.** Compile-path injection demands a standing threat-model review and a dedicated benchmark. **(2) Cost envelope.** Continuous compilation and continuous reasoning are LLM-heavy; cost levers such as model routing and incremental compilation need calibration on real workloads. **(3) Control and concurrency.** Agent activation and the orchestrator’s policy remain unspecified, and edits beyond the minimal regime of Sect. 4.1 exceed default Git merge semantics. **(4) Skill discovery at the long tail.** Novel goals need a compose-or-escalate path that the framework leaves under-specified. **(5) Empirical validation.** The position has not yet been benchmarked against the alternatives of Sect. 5, and the experiment that would test hypotheses H1 and H2 is specified in Sect. 7. None of these are blockers for stating the position; they are the work of testing it.

We offer the substrate-inversion position as work in progress: something to argue with and to test in deployment.

References

1. Challapally, A., et al.: The GenAI divide: State of AI in business 2025. MIT Project NANDA report (2025), https://web.archive.org/web/20260605074056/https://mlq.ai/media/quarterly_decks/v0.1_State_of_AI_in_Business_2025_Report.pdf, accessed: 2026-06-10

2. Chen, L., Zaharia, M., Zou, J.: How is ChatGPT’s behavior changing over time? *Harvard Data Science Review* **6**(2) (2024). <https://doi.org/10.1162/99608f92.5317da47>
3. Comanici, G., et al.: Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025)
4. Conway, M.E.: How do committees invent? *Datamation* **14**(4), 28–31 (1968)
5. Debenedetti, E., et al.: AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2024)
6. Debenedetti, E., et al.: Defeating prompt injections by design. In: *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)* (2026)
7. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024)
8. Erman, L.D., Hayes-Roth, F., Lesser, V.R., Reddy, D.R.: The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys* **12**(2), 213–253 (1980). <https://doi.org/10.1145/356810.356816>
9. Esteva, M., Rodríguez-Aguilar, J.A., Sierra, C., Garcia, P., Arcos, J.L.: On the formal specification of electronic institutions. In: *Agent Mediated Electronic Commerce. LNAI*, vol. 1991, pp. 126–147. Springer (2001)
10. Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.S., Li, Q.: A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. pp. 6491–6501 (2024). <https://doi.org/10.1145/3637528.3671470>
11. Gelernter, D.: Generative communication in Linda. *ACM Transactions on Programming Languages and Systems* **7**(1), 80–112 (1985). <https://doi.org/10.1145/2363.2433>
12. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. pp. 79–90 (2023). <https://doi.org/10.1145/3605764.3623985>
13. Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X.: Large language model based multi-agents: A survey of progress and challenges. In: *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. pp. 8048–8057 (2024)
14. Han, B., Zhang, S.: Exploring advanced LLM multi-agent systems based on black-board architecture. *arXiv preprint arXiv:2507.01701* (2025)
15. Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kiciman, E.: Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720* (2024)
16. Hou, X., Zhao, Y., Wang, S., Wang, H.: Model Context Protocol (MCP): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology* (2026). <https://doi.org/10.1145/3796519>
17. Hutchins, E.: *Cognition in the Wild*. MIT Press, Cambridge, MA (1995)
18. Karpathy, A.: llm-wiki: An LLM-maintained Markdown knowledge base. *GitHub Gist* (2026), <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>, accessed: 2026-06-09

19. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. *Computer* **36**(1), 41–50 (2003)
20. Kramer, J., Magee, J.: Self-managed systems: an architectural challenge. In: *Future of Software Engineering (FOSE 2007)*. pp. 259–268. IEEE (2007)
21. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. pp. 9459–9474 (2020)
22. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* **12**, 157–173 (2024)
23. Liu, X., et al.: AgentBench: Evaluating LLMs as agents. In: *International Conference on Learning Representations (ICLR)* (2024)
24. Maslej, N., et al.: Artificial intelligence index report 2025. arXiv preprint arXiv:2504.07139 (2025)
25. McMillan, D.: Structured context engineering for file-native agentic systems: Evaluating schema accuracy, format effectiveness, and multi-file navigation at scale. arXiv preprint arXiv:2602.05447 (2026)
26. Mialon, G., Fourier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T.: GAIA: A benchmark for General AI Assistants. In: *International Conference on Learning Representations (ICLR)* (2024)
27. Minsky, N.H., Ungureanu, V.: Law-governed interaction: a coordination and control mechanism for heterogeneous distributed systems. *ACM Trans. Softw. Eng. Methodol.* **9**(3), 273–305 (2000)
28. Nii, H.P.: The blackboard model of problem solving and the evolution of blackboard architectures. *AI Magazine* **7**(2), 38–53 (1986)
29. Omicini, A., Ricci, A., Viroli, M.: Artifacts in the A&A meta-model for multi-agent systems. *Autonomous Agents and Multi-Agent Systems* **17**(3), 432–456 (2008). <https://doi.org/10.1007/s10458-008-9053-x>
30. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560 (2023)
31. Parasuraman, R., Sheridan, T.B., Wickens, C.D.: A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A* **30**(3), 286–297 (2000)
32. Parnin, C., et al.: Building your own product copilot: Challenges, opportunities, and needs. In: *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. pp. 338–348 (2025)
33. Perez, F., Ribeiro, I.: Ignore previous prompt: Attack techniques for language models. In: *NeurIPS 2022 Workshop on Machine Learning Safety* (2022)
34. Salemi, A., et al.: LLM-based multi-agent blackboard system for information discovery in data science. arXiv preprint arXiv:2510.01285 (2025)
35. Scerri, P., Pynadath, D.V., Tambe, M.: Towards adjustable autonomy for the real world. *Journal of Artificial Intelligence Research* **17**, 171–228 (2002)
36. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2023)
37. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F., Dennison, D.: Hidden technical debt in machine learning

- systems. In: *Advances in Neural Information Processing Systems (NeurIPS)*. pp. 2503–2511 (2015)
38. Shavit, Y., et al.: Practices for governing agentic AI systems. OpenAI white paper (2023)
 39. Stauffer, L., Feng, K., Wei, K., Bailey, L., Duan, Y., Yang, M., Ozisik, A.P., Casper, S., Kolt, N.: The 2025 AI agent index: Documenting technical and safety features of deployed agentic AI systems. arXiv preprint arXiv:2602.17753 (2026)
 40. Theraulaz, G., Bonabeau, E.: A brief history of stigmergy. *Artificial Life* **5**(2), 97–116 (1999). <https://doi.org/10.1162/106454699568700>
 41. Vu, H., Klievtsova, N., Leopold, H., Rinderle-Ma, S., Kampik, T.: Agentic business process management: Practitioner perspectives on agent governance in business processes. In: *Business Process Management: Responsible BPM Forum, Process Technology Forum, Educators Forum (BPM 2025)*. LNBIP, vol. 565, pp. 29–43. Springer (2025). https://doi.org/10.1007/978-3-032-02936-2_3
 42. Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research* (2024)
 43. Weyns, D., Omicini, A., Odell, J.: Environment as a first class abstraction in multiagent systems. *Autonomous Agents and Multi-Agent Systems* **14**(1), 5–30 (2007). <https://doi.org/10.1007/s10458-006-0012-0>
 44. Wiederhold, G.: Mediators in the architecture of future information systems. *Computer* **25**(3), 38–49 (1992)
 45. Wooldridge, M., Jennings, N.R.: Intelligent agents: Theory and practice. *The Knowledge Engineering Review* **10**(2), 115–152 (1995). <https://doi.org/10.1017/S0269888900008122>
 46. Yao, S., Shinn, N., Razavi, P., Narasimhan, K.: τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In: *International Conference on Learning Representations (ICLR)* (2025)
 47. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)* (2023)